

Package ‘nutricalc’

May 3, 2026

Type Package

Title Nutrient Solution Formulation in Horticultural Sciences

Version 0.1.0

Description Provides tools for parsing chemical formulas, calculating molar masses, and optimizing fertilizer mixtures to meet specified nutrient targets using non-negative least squares. Designed to help agronomists and researchers formulate custom nutrient solutions efficiently.

License GPL-3

Encoding UTF-8

LazyData true

Depends R (>= 3.5)

Imports stringr,
nnls,
shiny,
bslib,
Ternary,
xml2

RoxygenNote 7.3.2

Contents

assign_salts_bak	2
CANONICAL_NUTRIENTS	3
compute_molar_mass	4
compute_percentage_matrix	4
converte	5
convert_units	5
drop_tiny	6
ec_from_ph	7
element_molar_mass_df	8
fetch_bwb_mittelwert	8
ion_diffusion_25C	9
nutrient_matrix	9
optimize_nutrients	10
percentage_matrix	10
ph_from_achieved	11
plot_ternary	12

print.bwb_mittelwert	13
print.nutrient_optimization_result	14
recipes	14
recipe_summary	15
run_app	15
salt_info	16
strip_negative_zero	16
two_stage_optimize_nutrients	17

Index	18
--------------	-----------

assign_salts_bak	<i>Assign salts to A, B, and Micro tanks based on composition and solver result</i>
------------------	---

Description

This helper takes the result from `optimize_nutrients()` and the corresponding `nutrient_matrix` (compounds x nutrients) and returns a BAK-style grouping:

Usage

```
assign_salts_bak(
  res,
  nutrient_matrix,
  micro_nutrients = c("Fe", "Mn", "Zn", "B", "Cu", "Mo", "Si"),
  calcium_nutrient = "Ca",
  phosphorus_nutrient = "P",
  sulfur_nutrient = "S",
  balance_nutrients = c("NO3_N", "NH4_N", "P", "K", "Ca", "Mg", "S", "Na"),
  drop_zero = TRUE,
  tol = 0
)
```

Arguments

<code>res</code>	A <code>nutrient_optimization_result</code> as returned by <code>optimize_nutrients()</code> .
<code>nutrient_matrix</code>	Matrix [compound x nutrient] used in the optimization. Row names must match the salt names (formulas).
<code>micro_nutrients</code>	Character vector of column names that count as micronutrients. Defaults to <code>c("Fe", "Mn", "Zn", "B", "Cu", "Mo")</code> .
<code>calcium_nutrient</code>	Column name for calcium (default: "Ca").
<code>phosphorus_nutrient</code>	Column name for phosphorus (default: "P").
<code>sulfur_nutrient</code>	Column name for sulfur (default: "S").
<code>balance_nutrients</code>	Character vector of nutrient columns used to define the "load" when splitting AB salts between A and B. Defaults to <code>c("NO3_N", "NH4_N", "P", "K", "Ca", "Mg", "S", "Na")</code> .

drop_zero	Logical, if TRUE (default), remove salts with effectively zero mmol/L from the returned table and by_tank. The assignments vector always includes all salts.
tol	Numeric tolerance for deciding what counts as "zero" mmol/L.

Details

- First, salts are classified purely by composition: * Micro = any micro nutrient present (e.g. Fe, Mn, Zn, B, Cu, Mo) * A = contains Ca (and no micro) * B = contains P or S (and no micro / Ca) * AB = none of the above (can go to A or B)

- Then the AB salts are assigned to A or B so that A and B have a similar overall "load" (sum of delivered mmol of selected nutrients).

Value

A list with:

assignments	Named character vector, salt -> tank ("A","B","Micro").
table	Data frame with columns salt, tank, mmol_l (filtered to non-zero if drop_zero = TRUE).
by_tank	Named list of data frames (A, B, Micro, possibly others), each subset of table.

CANONICAL_NUTRIENTS	<i>Canonical nutrient keys for targets and UI</i>
---------------------	---

Description

Defines the stable nutrient order used across recipes, UI inputs, and target vectors. This is intentionally independent from the full 'nutrient_matrix', which may include additional columns for chelate ligands or other species.

Usage

```
CANONICAL_NUTRIENTS
```

Format

A character vector of nutrient keys.

compute_molar_mass	<i>Compute molar mass of a chemical formula</i>
--------------------	---

Description

Supports formulas like "NaCl", "Ca(NO3)2", "CuSO4·5H2O", "MgN2O6·6H2O"

Usage

```
compute_molar_mass(formula)
```

Arguments

formula	Chemical formula string
---------	-------------------------

Value

Molar mass in g/mol

compute_percentage_matrix	<i>Compute a matrix of mass percentages of nutrients in each fertilizer</i>
---------------------------	---

Description

Compute a matrix of mass percentages of nutrients in each fertilizer

Usage

```
compute_percentage_matrix(nutrient_matrix)
```

Arguments

nutrient_matrix	A [fertilizer x nutrient] matrix in mmol/mol
-----------------	--

Value

A matrix of nutrient mass percentages per fertilizer

convert	<i>Convert amount of one compound to another using molar mass and element counts</i>
---------	--

Description

This function computes the conversion factor between two chemical formulas or elements based on their molar masses and the count of shared atoms. It adjusts for stoichiometry where applicable (e.g., converting from "SO4" to "S").

Usage

```
convert(from, to, value = 1, element = NULL)
```

Arguments

from	A character string representing the source formula or element (e.g., "Ca", "NaCl", "SO4").
to	A character string representing the target formula or element.
value	A numeric value to convert (default is 1). Represents the quantity of 'from'.
element	Optional element symbol to anchor the stoichiometric conversion when multiple elements are shared between 'from' and 'to'. If omitted and more than one element is shared, an error is raised to avoid ambiguous conversions.

Value

A numeric value representing the equivalent amount of 'to', adjusted by molar mass and atom ratio.

Examples

```
convert("SO4", "S")      # Convert sulfate to elemental sulfur
convert("Na", "NaCl")    # Convert sodium to sodium chloride
convert("P2O5", "P")     # Convert phosphorus pentoxide to elemental phosphorus
convert("Ca", "CaCl2\u00b72H2O", 10) # Convert 10 mmol of Ca to CaCl2\u00b72H2O
convert("(NH4)2HPO4", "NH4H2PO4", element = "P") # Disambiguate using P as the anchor
```

convert_units	<i>Convert nutrient concentrations between mmol/L and mg/L</i>
---------------	--

Description

Converts numeric nutrient values (single element or named vector) between mmol/L and mg/L using molar masses in 'element_molar_mass_df'.

Usage

```
convert_units(x, element = NULL, to = c("mg/L", "mmol/L"))
```

Arguments

x	Numeric scalar or named numeric vector of nutrient values. If named, names should match those in 'nutrient_element_map'.
element	Optional element symbol (e.g. "N", "P") when 'x' is a scalar.
to	Target unit: "mg/L" or "mmol/L".

Value

Numeric vector (same length/names as 'x') in the target unit.

Examples

```
convert_units(c(NO3_N = 15, P = 1.5), to = "mg/L")
convert_units(c(NO3_N = 210, P = 46), to = "mmol/L")
```

drop_tiny

Zero-out tiny floating point values

Description

Utility helper to clamp extremely small values to zero, avoiding noisy output.

Usage

```
drop_tiny(x, tol = 1e-09)
```

Arguments

x	Numeric vector or matrix.
tol	Values with absolute magnitude below this tolerance are set to zero.

Value

The input with near-zero entries replaced by zeros.

Examples

```
drop_tiny(c(1, 1e-10, -1e-12))
drop_tiny(matrix(c(0.5, 1e-12), nrow = 1))
```

ec_from_ph

*Compute EC from achieved composition and pH speciation***Description**

Estimates conductivity using an ion-transport summation based on diffusion coefficients and activity coefficients.

Usage

```
ec_from_ph(
  achieved,
  ph_res,
  fe_state = c("Fe2+", "Fe3+"),
  t_C = 25,
  gamma = NULL,
  gamma_model = 1,
  A_DH_25 = 0.5085
)
```

Arguments

achieved	Named numeric vector (mmol/L) or data.frame with columns Nutrient and Achieved.
ph_res	Result list from 'ph_from_achieved()' containing 'species_mM'.
fe_state	One of "Fe2+" or "Fe3+". When "Fe3+", all Fe2+ is treated as Fe3+ for EC purposes.
t_C	Temperature in degrees C.
gamma	Optional named numeric vector of activity coefficients (per ion).
gamma_model	Activity model selector: 1/"davies_25C" (Davies) or 2/"debye_huckel_25C" (Debye-Huckel). Default 1.
A_DH_25	Debye-Huckel A constant at 25 C (also used as A in Davies).

Details

Units: - c is in mmol/L (mM); numerically identical to mol/m³, used directly as such. - D is in 10⁻⁵ cm²/s and converted to m²/s by multiplying 1e-9. - Output: EC in mS/cm and uS/cm.

gamma_model: 1 = Davies (default) 2 = Debye-Hückel

Override precedence: model baseline -> ph_res\$gamma_ions -> gamma argument (highest priority)

Value

A list with EC_mS_cm, EC_uS_cm, and per-ion contributions.

element_molar_mass_df *Atomic molar masses used for formula parsing*

Description

A lookup table of elemental (atomic) molar masses in grams per mole (g/mol). Used by molar-mass and unit-conversion helpers (e.g., when computing the molar mass of a chemical formula from its elemental composition).

Usage

```
data(element_molar_mass_df)
```

Format

A data frame with 23 rows and 2 columns:

Element Element symbol (character), e.g. "C", "O", "Fe".

MolarMass_g_per_mol Atomic molar mass in g/mol (numeric).

fetch_bwb_mittelwert *Fetch BWB "In aller Tiefe" Mittelwerte by postal code*

Description

Retrieves Berliner Wasserbetriebe (BWB) analysis data for a given Berlin postal code and returns the Mittelwert values (first available entry per parameter) for selected parameters as mmol/L.

Usage

```
fetch_bwb_mittelwert(plz)
```

Arguments

plz Character or numeric vector of length one with a five digit German postal code.

Value

A named numeric vector of length 18 in mmol/L with names 'c("NO3_N", "NH4_N", "P", "K", "Ca", "Mg", "S", "Na", "Cl", "Fe", "Mn", "Zn", "B", "Cu", "Mo", "Si", "KS4_3", "KB8_2")', limited to the decimal precision provided by the source values. 'KS4_3' is the acid capacity (alkalinity, KS 4.3); 'KB8_2' is the base capacity (acidity, KS 8.2) and is treated as dissolved CO₂(aq).

ion_diffusion_25C	<i>Lookup table of ion diffusion coefficients at 25 C</i>
-------------------	---

Description

Returns a data frame with ions, charge, and diffusion coefficients (D) at 25 C. Values are expressed as $10^{-5} \text{ cm}^2/\text{s}$.

Usage

```
ion_diffusion_25C()
```

Value

A data.frame with columns: ion, z, D_1e5_cm2_s.

nutrient_matrix	<i>Canonical fertilizer-to-nutrient stoichiometry matrix</i>
-----------------	--

Description

A compound-by-nutrient matrix used by the nutrient solvers. Rows are fertilizer compounds (salts/acids/bases) and columns are nutrients. Each entry is the amount of nutrient delivered by 1 mmol of the compound (mmol nutrient per mmol compound).

Usage

```
data(nutrient_matrix)
```

Format

A numeric matrix with n rows (compounds) and 20 columns (nutrients). Row names are chemical formulas; column names are nutrient keys: 'NO3_N', 'NH4_N', 'P', 'K', 'Ca', 'Mg', 'S', 'Na', 'Cl', 'Fe', 'Mn', 'Zn', 'B', 'Cu', 'Mo', 'Si', 'EDTA', 'DTPA', 'EDDHA', 'HBED'.

Details

The matrix entries are stoichiometric coefficients in mmol/mmol. For example, calcium nitrate ('Ca(NO3)2') contributes 2 mmol 'NO3_N' and 1 mmol 'Ca' per 1 mmol of salt.

optimize_nutrients	<i>Optimize Nutrient Solution Using NNLS</i>
--------------------	--

Description

Solves for optimal fertilizer amounts to meet target nutrient concentrations. Expects a compound-by-nutrient matrix: rows = salts/compounds, columns = nutrients.

Usage

```
optimize_nutrients(nutrient_matrix, target, importance)
```

Arguments

nutrient_matrix	Matrix [compound x nutrient] with entries in mol of nutrient per mmol of compound (i.e., how many mmol of nutrient you get from 1 mmol of the compound).
target	Named numeric vector of target concentrations (mmol/L).
importance	Named numeric vector (-2 to +2) specifying nutrient importance (weights).

Value

An object of class 'nutrient_optimization_result' with elements:

- amounts - named numeric (mmol/L) of each compound (same order as rownames of input)
- achieved - named numeric (mmol/L) delivered per nutrient
- target - named numeric (mmol/L) target per nutrient
- abs_error - achieved - target (mmol/L)
- percent_error - 100 * abs_error / target (NA when target == 0)
- squared_error - sum(abs_error^2)
- rel_squared_error - sum((abs_error/target)^2, na.rm = TRUE)

percentage_matrix	<i>Nutrient mass percentage matrix</i>
-------------------	--

Description

Mass percentages of nutrients in each fertilizer derived from 'nutrient_matrix'.

Usage

```
data(percentage_matrix)
```

Format

A numeric matrix with rows as fertilizer formulas and columns as nutrients (NO3_N, NH4_N, P, K, Ca, Mg, S, Na, Cl, Fe, Mn, Zn, B, Cu, Mo, Si). Each value is the mass percentage of the nutrient in the fertilizer.

ph_from_achieved	<i>Compute pH of a nutrient solution from achieved composition</i>
------------------	--

Description

Uses charge balance with acid/base speciation and activity corrections to estimate pH at 25 C. Assumes no carbonate alkalinity, no complexation, and no precipitation.

Usage

```
ph_from_achieved(
  achieved,
  temp_C = 25,
  phc_bracket = c(3, 9),
  tol = 1e-09,
  max_iter = 200,
  inner_max_iter = 50,
  debug = FALSE,
  chelate_z = c(EDTA = -4, DTPA = -5, EDDHA = -4, HBED = -4),
  gamma_model = 1
)
```

Arguments

achieved	Named numeric vector of mmol/L (e.g., solver result 'x\$sachieved'). If present, 'CO2_aq' is treated as dissolved free CO2(aq) (mmol/L), mapped from BWB "Basenkapazitaet KB 8,2" / KS 8.2 and used to fix carbonate speciation.
temp_C	Temperature in Celsius (only 25 C supported in this version).
phc_bracket	Numeric length-2 bracket for pHc = -log10([H+]) search.
tol	Root tolerance in meq/L.
max_iter	Maximum iterations for the outer bisection.
inner_max_iter	Maximum iterations for the ionic strength fixed point.
debug	Logical; when TRUE prints basic bracket diagnostics.
chelate_z	Named numeric vector of chelate charges to include as fixed anions. Values are net charges (negative) for ligand totals in mmol/L.
gamma_model	Integer 1 (Davies) or 2 (Debye–Hückel). Default 1.

Details

gamma_model: 1 = Davies (default) 2 = Debye–Hückel limiting law (DH)

Value

A list with pH, ionic strength, activity coefficients, species distribution, and charge balance diagnostics.

plot_ternary	<i>Steiner-style ternary plot (Target vs Achieved) with constant-component drops</i>
--------------	--

Description

Steiner-style ternary plot (Target vs Achieved) with constant-component drops

Usage

```
plot_ternary(
  result,
  keys,
  title = "Ternary (ratios)",
  show_legend = TRUE,
  step_percent = 10,
  lab_cex = 0.95,
  axis_cex = 0.85,
  padding = 0.08,
  col_target = "black",
  col_achieved = "firebrick",
  pch_target = 16,
  pch_achieved = 17,
  cex_points = 1.25,
  show_link = FALSE,
  lty_link = 2,
  lwd_link = 1,
  legend_pos = "topright",
  show_axis_drops = TRUE,
  drop_col = "grey35",
  drop_lwd = 1,
  drop_lty = 1,
  show_ratio = TRUE,
  ratio_format = "percent",
  ratio_digits = 1
)
```

Arguments

result	List with numeric named vectors 'target' and 'achieved'.
keys	Character(3) giving axis order, e.g. c("K", "Ca", "Mg").
title	Plot title.
show_legend	Logical, show legend (default TRUE).
step_percent	Integer in [1, 50]; major grid step as percent. Default 10.
lab_cex, axis_cex, padding	Plot layout controls.
col_target, col_achieved	Colors.
pch_target, pch_achieved	Point symbols.

cex_points	Point size.
show_link	Logical, draw connector between Target and Achieved.
lty_link, lwd_link	Style for connector if shown.
legend_pos	Legend position.
show_axis_drops	Logical, draw constant-component drops from Achieved to sides.
drop_col, drop_lwd, drop_lty	Style for drop lines.
show_ratio	Logical, include "(A, B; C)" values in legend (default TRUE).
ratio_format	"percent" (default) or "proportion".
ratio_digits	Digits for numeric formatting.

Value

Invisibly returns NULL (draws the plot).

```
print.bwb_mittelwert  Pretty-print BWB Mittelwerte
```

Description

Pretty-print BWB Mittelwerte

Usage

```
## S3 method for class 'bwb_mittelwert'
print(x, ...)
```

Arguments

x	A 'bwb_mittelwert' numeric vector.
...	Additional arguments passed to 'print'.

Value

The input 'bwb_mittelwert' object, invisibly.

```
print.nutrient_optimization_result
```

Pretty-print the optimization result

Description

Shows per-liter amounts; if 'vol > 1', also shows a dynamic column "g for <vol> L". Uses 'compute_molar_mass(formula)' (g/mol) when available; if not found, prints without mass columns.

Usage

```
## S3 method for class 'nutrient_optimization_result'
print(x, vol = 1, ...)
```

Arguments

x	A 'nutrient_optimization_result' object.
vol	Total solution volume in liters (must be > 0). If > 1, a "g for <vol> L" column is added.
...	Unused.

```
recipes
```

Nutrient solution recipes used by NutriCalc

Description

A named list of recipe definitions. Each element is a list with:

name Display name

category Category such as "Olericulture", "Fruticulture", ...

notes Free text notes / citation

unit Either "mmol/L" or "mg/L"

salts TRUE (all salts) or a character vector of salt formulas

targets Named numeric vector of nutrient targets

Usage

```
recipes
```

Format

A named list of recipe lists.

`recipe_summary`*Create a summary table of nutrient concentrations in both units*

Description

Given a named vector of nutrient concentrations and its unit, returns a data frame showing each nutrient's value in mmol/L and mg/L.

Usage

```
recipe_summary(recipe, unit = c("mmol/L", "mg/L"))
```

Arguments

<code>recipe</code>	Named numeric vector of nutrient concentrations (e.g., NO3_N, P, K, ...).
<code>unit</code>	Current unit of the values: "mmol/L" or "mg/L".

Value

A data frame with columns 'Nutrient', 'Element', 'mmol/L', 'mg/L'.

Examples

```
recipe_summary(c(NO3_N = 15, P = 1.5), unit = "mmol/L")
recipe_summary(c(NO3_N = 210, P = 46), unit = "mg/L")
```

`run_app`*Launch the NutriCalc Shiny App*

Description

Launch the NutriCalc Shiny App

Usage

```
run_app(...)
```

Arguments

<code>...</code>	Arguments passed to <code>[shiny::runApp()]</code> .
------------------	--

`salt_info`*Metadata for fertilizer compounds*

Description

Descriptive metadata for each compound in `nutrient_matrix`, including a human-readable category name and whether the compound is treated as an acid/base in the two-stage solver.

Usage

```
data(salt_info)
```

Format

A data frame with n rows and 3 columns:

salt Chemical formula key matching `rownames(nutrient_matrix)`.

category Human-readable compound name/category.

is_acid_base Logical flag; TRUE for acids/bases.

`strip_negative_zero`*Remove negative zeros introduced by rounding*

Description

Ensures rounded values like -0.00 are displayed as 0.

Usage

```
strip_negative_zero(x)
```

Arguments

x Numeric vector or matrix.

`two_stage_optimize_nutrients`*Two-stage nutrient optimization separating neutral salts from acids/bases*

Description

Runs an initial NNLS optimization on neutral salts, then uses acids/bases to fill remaining deficiencies (when they can affect the off-target nutrients).

Usage

```
two_stage_optimize_nutrients(  
  nutrient_matrix,  
  target,  
  importance,  
  tol_abs = 0.01,  
  tol_rel = 0.01  
)
```

Arguments

<code>nutrient_matrix</code>	Matrix [compound x nutrient] with entries in mol of nutrient per mmol of compound.
<code>target</code>	Named numeric vector of target concentrations (mmol/L).
<code>importance</code>	Named numeric vector (-2 to +2) specifying nutrient importance (weights).
<code>tol_abs</code>	Absolute tolerance (mmol/L) below which residuals are ignored for acid/base correction.
<code>tol_rel</code>	Relative tolerance (fraction of target) below which residuals are ignored for acid/base correction.

Value

A 'nutrient_optimization_result' object.

Index

* datasets

- CANONICAL_NUTRIENTS, [3](#)
- element_molar_mass_df, [8](#)
- nutrient_matrix, [9](#)
- recipes, [14](#)
- salt_info, [16](#)

assign_salts_bak, [2](#)

CANONICAL_NUTRIENTS, [3](#)
compute_molar_mass, [4](#)
compute_percentage_matrix, [4](#)
convert_units, [5](#)
converte, [5](#)

drop_tiny, [6](#)

ec_from_ph, [7](#)
element_molar_mass_df, [8](#)

fetch_bwb_mittelwert, [8](#)

ion_diffusion_25C, [9](#)

nutrient_matrix, [9](#), [16](#)

optimize_nutrients, [10](#)

percentage_matrix, [10](#)
ph_from_achieved, [11](#)
plot_ternary, [12](#)
print.bwb_mittelwert, [13](#)
print.nutrient_optimization_result, [14](#)

recipe_summary, [15](#)
recipes, [14](#)
run_app, [15](#)

salt_info, [16](#)
strip_negative_zero, [16](#)

two_stage_optimize_nutrients, [17](#)